

Mixed-Precision Quantization for Deep Vision Models with Integer Quadratic Programming

Zihao Deng*
The University of Texas at Austin
zihaodeng@utexas.edu

Sayeh Sharify*
d-Matrix Corp.
sayehs@d-matrix.ai

Xin Wang
d-Matrix Corp.
xwang@d-matrix.ai

Michael Orshansky
The University of Texas at Austin
orshansky@utexas.edu

Abstract—Quantization is a widely used technique to compress neural networks. Assigning uniform bit-widths across all layers can result in significant accuracy degradation at low precision and inefficiency at high precision. Mixed-precision quantization (MPQ) addresses this by assigning varied bit-widths to layers, optimizing the accuracy-efficiency trade-off. Existing sensitivity-based methods for MPQ assume that quantization errors across layers are independent, which leads to suboptimal choices. We introduce CLADO, a practical sensitivity-based MPQ algorithm that captures cross-layer dependency of quantization error. CLADO approximates pairwise cross-layer errors using linear equations on a small data subset. Layerwise bit-widths are assigned by optimizing a new MPQ formulation based on cross-layer quantization errors using an Integer Quadratic Program. Experiments with CNN and vision transformer models on ImageNet demonstrate that CLADO achieves state-of-the-art mixed-precision quantization performance. Code repository available [here](https://github.com/JamesTuna/CLADO_MPQ).¹

I. INTRODUCTION

Reducing the storage and computational demands of deep neural networks (DNNs) is essential for practical applications. Model quantization effectively compresses DNNs [5, 6, 16, 22, 29] and, when applied to pretrained models, is known as post-training quantization (PTQ). PTQ typically requires only a small dataset to calibrate activation and weight statistics, allowing efficient determination of optimal quantization ranges for each layer. This calibration process is both data-efficient and quick to deploy, with bit-widths specified for each layer.

Uniform precision quantization (UPQ), the simplest quantization approach, assigns the same bit-width to all layers but overlooks layer-specific tolerance to quantization. However, research has shown that certain layers are more robust to quantization than others [2, 3, 10]; while quantizing some layers to low bit-width may yield minimal performance loss, even moderate quantization of other layers can lead to significant degradation. To take advantage of this phenomenon, mixed precision quantization (MPQ) optimizes bit-width assignments across layers such that lower prediction error is achieved at the same target compression rate as UPQ. Naïvely formulated, the search space of MPQ grows *exponentially* with the number of layers (L) to quantize. To address this, prior work presented two classes of solutions: *search-based methods* and *sensitivity-based methods* [27].

Search-based methods adopt search processes in which a large number of bit-width assignments are tested [1, 8, 12, 20, 28, 31]. These search-based methods are usually expensive, hard to parallelize, and will take hundreds or even thousands of GPU hours because of their iterative search nature. Sensitivity-based methods, on the other hand, evaluate a closed-form metric that measures the tolerance of layers to quantization [2, 3, 4, 9, 17, 21, 27, 32]. Such a metric, *viz.* *sensitivity*, is a property of each quantized layer. Various formulations have been proposed for the sensitivity metric in practice.

Despite the diversity in the formulation of sensitivity, all sensitivity-based MPQ methods minimize total sum of sensitivities across layers, constrained by a target compression ratio. Such an optimization problem is much more efficient to solve. These methods take several minutes to a few GPU-hours to complete the entire MPQ optimization process. Importantly, their runtime can be further reduced by parallel computation of sensitivities and these sensitivities can be reused when requirements change. *Our work is also sensitivity-based.*

A key limitation of existing sensitivity-based MPQ algorithms is their assumption of independent layer-wise quantization effects, or dependence only within local blocks, as seen in BRECQ [18]. This assumption allows for an objective function that is simple to formulate and optimize but overlooks the interactions between quantized layers—what we refer to as the *cross-layer dependency of quantization error*. This oversight is especially impactful in computer vision models, where we demonstrate that an MPQ solution accounting for these dependencies outperforms methods assuming layer independence.

To overcome this limitation, we propose CLADO (Cross-Layer-Dependency-aware Optimization), an algorithm that measures the cross-layer dependency for all layers efficiently and optimizes a new cross-layer dependency aware objective via Integer Quadratic Programming (IQP). Following prior work, we start by applying the second-order Taylor expansion on network quantization loss. Then, we decompose the loss into two components with each being a sum of terms. Terms in the first component are specific to individual layers and they act independently across layers. They are well-addressed in prior work and we refer to them as the *layer-specific sensitivities*. Terms in the second component, which prior work ignores, represent interactions between layers and we refer to them as the *cross-layer sensitivities*. Naïvely evaluating them requires calculation of inter-layer Hessian matrices, which is practically infeasible. We propose an efficient backpropagation-free method that avoids Hessian calculation and computes sensitivities by solving a set of linear equations that requires $O(L^2)$ times evaluations of the network on a small set of samples. These sensitivities then allow us to reformulate the original MPQ problem as an IQP that contains $O(L)$ binary variables with linear constraints. The contributions of our work are:

- CLADO, the first sensitivity-based algorithm that captures the interactions of quantization error across *all layers* and transforms the MPQ problem into an *Integer Quadratic Program* that can be solved within seconds.
- CLADO is enabled by an efficient method to compute the cross-layer dependencies in $O(L^2)$ time. It is backpropagation-free and only requires forward evaluations of DNNs on a small amount of training data.
- Experiments on computer vision models, including ResNets, MobileNets, and ViT, show that CLADO achieves state-of-the-art mixed-precision performance.

*These authors contributed equally to this work.

¹https://github.com/JamesTuna/CLADO_MPQ

II. PRIOR WORK

Existing mixed-precision quantization techniques can be divided into two classes: search-based and sensitivity based.

Search-based methods: These methods evaluate the performance of mixed-precision quantized networks and use it to guide a search process. HAQ [28] and AutoQ [20] use Reinforcement Learning (RL), where all possible bit-width assignments of layers form the action space and the MPQ performance acts as the reward. Other search methods, such as MPQDNAS [31] and SPOS [12], apply Neural Architecture Search (NAS) to make MPQ a differentiable search process. The advantage of these methods is that they make decisions based on explicit evaluation of the quantized model’s loss. However, they are computationally demanding, usually taking hundreds or even thousands of GPU-hours to complete [27]. Moreover, because each search is tied to a fixed set of constraints, when requirements change, new search processes need to be launched from scratch for search-based methods.

Sensitivity-based methods: These methods pre-compute *layer sensitivity* as a proxy metric (or “critic”) to assess the impact of quantization on model performance. The sensitivities of layers in certain quantization precision are usually fast to measure, and once measured, they are reused in estimating the optimization objective as a function of different combinations of layer-wise precision assignments. This type of methods, including ZeroQ [2], variants of HAWQ [9, 10] and MPQCO [4], formulates the MPQ as a constrained combinatorial optimization problem. The objective to be minimized is the sum of layer sensitivities that quantify the overall quantization impact, while the constraints ensure certain target compression requirements, such as memory and/or computational budgets. Other works, such as OMPQ [21], BRECO [18], and Bisection sensitivity-search [3], also fall into the category of sensitivity-based methods and propose different approaches for measuring layer sensitivity. For layer sensitivities defined in some of the above mentioned work, EMQ [8] studied the consistency between them and the actual network performance. We note that there is also prior work, BRECO [18], that defines sensitivities on blocks of adjacent layers. BRECO shares the same methodologies as the aforementioned approach in formulating the MPQ as an optimization problem with its objective being the summation of sensitivities and its constraints being the compression requirement.

To measure the layer sensitivities, a small subset of training data is needed (e.g., 1024 training samples for ImageNet). We refer to it as the *sensitivity set*. Since sensitivity-based methods optimize a proxy, instead of the real increase in network loss, they do not require iterative model evaluation and are efficient to solve. State-of-the-art sensitivity-based algorithms achieve performance comparable to that of search-based algorithms [4]. *Our proposed method CLADO is also sensitivity-based.*

III. CROSS-LAYER DEPENDENCY & MPQ OPTIMALITY

Empirically, cross-layer dependencies play a non-negligible role in the search of optimal MPQ solutions and ignoring them may lead to suboptimal solutions. Here we present, on ImageNet, two examples of MPQ suboptimality caused by ignoring the cross-layer dependencies. To make the suboptimality easy to understand, we focus on only a few layers and the goal is to *choose two layers to quantize so that the quantization loss is minimized.*

Figure 1a shows the sensitivity matrix of ResNet-34 under 2-bit quantization for four specific layers: layer2.2.conv2 (index 12), layer3.1.conv2 (index 19), layer3.3.conv1 (index 22), and layer3.3.conv2 (index 23). Ignoring the cross-layer interactions

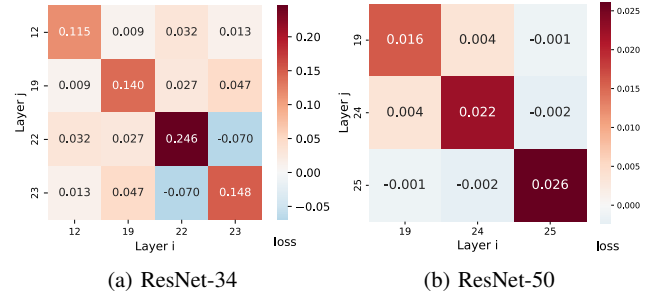


Fig. 1: Sensitivity matrix of ResNet models. Entry (i, i) : increase in loss caused by quantizing a single layer i . Entry (i, j) : extra increase in loss when quantizing a pair of layers (i, j) , compared to individually quantizing them.

(off-diagonal terms) leads to the selection of layers (12, 19) because it gives the smallest change in predicted loss $0.115 + 0.140 = 0.255$. However, the real induced loss change of quantizing layers (12, 19) is $0.115 + 0.140 + 2 \times 0.009 = 0.273$ and the optimal solution is to quantize layers (22, 23) because it has the smallest induced change in loss $0.246 + 0.148 + 2 \times (-0.070) = 0.254$. Similarly, Figure 1b shows the sensitivity matrix of ResNet-50 under 4-bit quantization for three layers: layer2.2.conv3 (index 19), layer3.0.conv2 (index 24), and layer3.0.conv3 (index 25). Ignoring cross-layer interactions would lead to selecting layers (19, 24), as this combination yields the smallest predicted loss change, calculated as $0.016 + 0.022 = 0.038$. However, the actual induced loss change is $0.038 + 2 \times 0.004 = 0.046$. The optimal solution instead is to quantize layers (19, 25), yielding the smallest induced loss change of $0.016 + 0.026 + 2 \times (-0.001) = 0.040$.

IV. CLADO: CROSS LAYER DEPENDENCY AWARE OPTIMIZATION

A. Preliminaries

We first introduce the formulation of the MPQ problem following Chen et al. [4].

Notation: We assume an I -layer (indices start from 0) neural network $f : \Theta \times \mathbb{X} \rightarrow \mathbb{Y}$ and a training dataset of N samples $(\mathbf{x}^{(n)}, \mathbf{y}^{(n)}) \in \mathbb{X} \times \mathbb{Y}$ with $n \in \{1, \dots, N\}$. The model maps each sample $\mathbf{x}^{(n)}$ to a prediction $\hat{\mathbf{y}}^{(n)} = f(\theta, \mathbf{x}^{(n)})$, using some parameters $\theta \in \Theta$. Then the predictions are compared with the ground truth $\mathbf{y}^{(n)}$ and evaluated with a task-specific loss function $\ell : \mathbb{Y} \times \mathbb{Y} \rightarrow \mathbb{R}$. This leads to the objective function to minimize $\mathcal{L} : \Theta \rightarrow \mathbb{R}$:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(f(\theta, \mathbf{x}^{(n)}), \mathbf{y}^{(n)}) = \frac{1}{N} \sum_{n=1}^N \ell^{(n)}(\theta) \quad (1)$$

We denote the weight tensor of the i^{th} layer as $W^{(i)} \in \mathbb{R}^{c_o \times c_{in} \times k \times k}$ and its flattened version as $w^{(i)} \in \mathbb{R}^{c_o c_{in} k^2}$, where k is the kernel size for a convolutional layer and $k = 1$ for fully-connected layers, c_{in} and c_o are the number of input and output channels, respectively. The quantization function is denoted by $Q : \mathbb{R}^D \times \mathbb{Z}^+ \rightarrow \Pi_b$, which takes a full-precision vector and the quantization bit-width as input and produces the quantized vector. Unless otherwise stated, we used per-tensor uniform symmetric quantization as it is the simplest scheme in practice due to its ease of implementation in hardware. As a result, $\Pi_b = s \times \{-2^{b-1}, \dots, 0, \dots, 2^{b-1} - 1\}$ for signed input and $s \times \{0, \dots, 2^b - 1\}$ for unsigned one, where b is the quantization bit-width and s is the quantization scale factor. The quantized b -bit

weight is given by $Q(w, b) = \text{clip}(\lfloor w/s \rfloor, -2^{b-1}, 2^{b-1} - 1) \times s$ for full precision w .

Discrete constrained optimization formulation: Let $w \triangleq \{w^{(i)}\}_{i=0}^{I-1}$ be the set of flattened weights of the network with I layers. To find the optimal bit-width assignment with the goal of minimizing total model size, MPQ can be written as the following discrete constrained problem. $\mathbb{B} = \{b_1, b_2, \dots, b_{|\mathbb{B}|}\}$ denotes the set of bit-widths to be assigned in MPQ. In this work, C_{target} is the target model size of the network, and $|\cdot|$ denotes the length of vectors.

$$\begin{aligned} \min_{\{b^{(i)}\}} & \frac{1}{N} \sum_{n=1}^N \ell(f(w + \Delta w, \mathbf{x}^{(n)}), \mathbf{y}^{(n)}) \\ \text{s.t.} & \Delta w^{(i)} = Q(w^{(i)}, b^{(i)}) - w^{(i)} \\ & \sum_i |w^{(i)}| \cdot b^{(i)} \leq C_{\text{target}} \\ & b^{(i)} \in \mathbb{B}, i \in \{0, \dots, I-1\} \end{aligned} \quad (2)$$

Proxy of optimization objective: Let $g_w \triangleq \nabla \mathcal{L}(w)$ and $H_w \triangleq \nabla^2 \mathcal{L}(w)$ be the gradient and the Hessian, respectively. For computational tractability, CLADO approximates the objective via the second-order Taylor expansion:

$$\begin{aligned} \mathcal{L}(w + \Delta w) &= \frac{1}{N} \sum_{n=1}^N \ell^{(n)}(w + \Delta w) \\ &\approx \mathcal{L}(w) + g_w^T \Delta w + \frac{1}{2} \Delta w^T H_w \Delta w \end{aligned} \quad (3)$$

For a well-trained model, it is reasonable to assume that training has converged to a local minimum, and thus, the gradient $g_w \approx 0$, resulting in:

$$\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + \frac{1}{2} \Delta w^T H_w \Delta w \quad (4)$$

In essence, MPQ seeks to minimize $\mathcal{L}(w + \Delta w)$ as a function of Δw , treating $\mathcal{L}(w)$ as a constant; thus, minimizing $\mathcal{L}(w + \Delta w)$ is equivalent to minimizing $\frac{1}{2} \Delta w^T H_w \Delta w$. Here we use $\Delta w^T H_w \Delta w$ as a proxy of the objective in (2).

B. Methods

The core of CLADO is a way to capture in full the pairwise interactions of quantization error between all layers, which is either ignored, or only partially addressed in prior work.

Layer-specific and cross-layer sensitivities: Let H_{ij} s be the partition of the Hessian at layer-level granularity. Specifically, let $H_{ii} \triangleq \partial^2 \mathcal{L}(w) / \partial w^{(i)2}$ be the Hessian of layer i and $H_{ij} \triangleq \partial^2 \mathcal{L}(w) / \partial w^{(i)} \partial w^{(j)}$ be the cross-layer Hessian between layers i and j . Unless otherwise stated, we use i, j to denote the indices of layers and they take values from $\{0, 1, \dots, I-1\}$, where I is the number of layers in the network. Then,

$$\Delta w^T H_w \Delta w = \sum_i \Delta w^{(i)T} H_{ii} \Delta w^{(i)} + \sum_{i \neq j} \Delta w^{(i)T} H_{ij} \Delta w^{(j)} \quad (5)$$

Equation 5 decomposes the objective into two terms. The first term captures the contribution to loss increase from individual layers due to the intra-layer effects resulted from quantization. We refer to them as *layer-specific sensitivities*. The second part contains the contributions from effects of pairwise layer interaction as a consequence of two different layers being jointly quantized. We refer to them as the *cross-layer sensitivities*. We use $\Omega_{i,j}(\cdot, \cdot)$ as a unified notation for both types of sensitivities.

$$\Omega_{i,j}(\Delta w^{(i)}, \Delta w^{(j)}) \triangleq \Delta w^{(i)T} H_{ij} \Delta w^{(j)} \quad (6)$$

We rewrite (5) in terms of Ω as follows; minimizing (5) is equivalent to minimizing:

$$\Omega = \sum_i \Omega_{i,i}(\Delta w^{(i)}, \Delta w^{(i)}) + \sum_{i \neq j} \Omega_{i,j}(\Delta w^{(i)}, \Delta w^{(j)}) \quad (7)$$

Note that this re-definition obviates direct reference to the Hessian. We will show below that $\Omega_{i,j}$ can be evaluated without computing the Hessian. Unlike prior work [2, 4, 9, 10] that ignores the cross-layer interaction terms, here we optimize (7) in its full form:

$$\begin{aligned} \min_{\{b^{(i)}\}} & \Omega \\ \text{s.t.} & \Delta w^{(i)} = Q(w^{(i)}, b^{(i)}) - w^{(i)} \\ & \sum_i |w^{(i)}| \cdot b^{(i)} \leq C_{\text{target}} \\ & b^{(i)} \in \mathbb{B}, i \in \{0, \dots, I-1\} \end{aligned} \quad (8)$$

IQP formulation: Next we show that (8) can be formulated as an Integer Quadratic Program (IQP) that solves for the bit-width assignment decisions. Since each layer i has $|\mathbb{B}|$ candidate bit-width choices: $\{b_1, b_2, \dots, b_{|\mathbb{B}|}\}$, $\Delta w^{(i)}$ can take $|\mathbb{B}|$ values: $\Delta w^{(i)} \in \{\Delta w_1^{(i)}, \Delta w_2^{(i)}, \dots, \Delta w_{|\mathbb{B}|}^{(i)}\}$. Here, we use $\Delta w_m^{(i)} \triangleq Q(w^{(i)}, b_m) - w^{(i)}$ with $m \in \{1, 2, \dots, |\mathbb{B}|\}$ to denote the quantization error on $w^{(i)}$, when quantizing it to b_m bits. For each layer i , we introduce a one-hot variable $\alpha^{(i)} \in \{0, 1\}^{|\mathbb{B}|}$ s.t. $\sum_{m=1}^{|\mathbb{B}|} \alpha_m^{(i)} = 1$ to represent this layer's bit-width decision. The single entry of 1 in $\alpha^{(i)}$, e.g. $\alpha_m^{(i)}$, indicates the selection of a corresponding $\Delta w_m^{(i)}$ and the chosen bit-width b_m . For compact notation, let $\alpha \in \mathbb{R}^{|\mathbb{B}|I}$ be the concatenation of $\alpha^{(i)}$ s, i.e., $\alpha = (\alpha^{(0)}, \alpha^{(1)}, \dots, \alpha^{(I-1)})$. Specifically,

$$\begin{aligned} \Delta w^{(i)} &= \alpha_1^{(i)} \Delta w_1^{(i)} + \alpha_2^{(i)} \Delta w_2^{(i)} + \dots + \alpha_{|\mathbb{B}|}^{(i)} \Delta w_{|\mathbb{B}|}^{(i)} \\ \Delta w_m^{(i)} &= Q(w^{(i)}, b_m) - w^{(i)}, m \in \{1, 2, \dots, |\mathbb{B}|\} \\ \alpha_1^{(i)}, \alpha_2^{(i)}, \dots, \alpha_{|\mathbb{B}|}^{(i)} &\in \{0, 1\}, \alpha_1^{(i)} + \alpha_2^{(i)} + \dots + \alpha_{|\mathbb{B}|}^{(i)} = 1 \end{aligned} \quad (9)$$

In addition, we gather all $\Omega_{i,j}$ s into a matrix $\hat{G} \in \mathbb{R}^{|\mathbb{B}|I \times |\mathbb{B}|I}$, to which we refer to as the *sensitivity matrix*. Specifically, for $m, n \in \{1, 2, \dots, |\mathbb{B}|\}$:

$$\hat{G}_{|\mathbb{B}|i+m, |\mathbb{B}|j+n} \triangleq \Omega_{i,j}(\Delta w_m^{(i)}, \Delta w_n^{(j)}) \quad (10)$$

Expanding the $\Delta w^{(i)}, \Delta w^{(j)}$ terms in (5) by (9) and bringing in the definition of $\hat{G}$ in (10), we have $\Omega = \Delta w^T H_w \Delta w = \alpha^T \hat{G} \alpha$. Accordingly, the MPQ problem is equivalent to the following IQP:

$$\begin{aligned} \min_{\{\alpha^{(i)}\}} & \alpha^T \hat{G} \alpha \\ \text{s.t.} & \alpha = (\alpha^{(1)}, \alpha^{(2)}, \dots, \alpha^{(I)}) \\ & \alpha_1^{(i)}, \alpha_2^{(i)}, \dots, \alpha_{|\mathbb{B}|}^{(i)} \in \{0, 1\}, \alpha_1^{(i)} + \alpha_2^{(i)} + \dots + \alpha_{|\mathbb{B}|}^{(i)} = 1 \\ & \sum_i \sum_m \alpha_m^{(i)} |w^{(i)}| \cdot b_m \leq C_{\text{target}} \\ & i \in \{0, \dots, I-1\}, m \in \{1, \dots, |\mathbb{B}|\} \end{aligned} \quad (11)$$

Backpropagation-free sensitivity measurement: We now present an efficient way to compute $\hat{G}$ through only forward evaluations of DNNs. For any pair of layers i, j and their corresponding bit-width choices m, n , computing $\Omega_i(\Delta w_m^{(i)})$, $\Omega_j(\Delta w_n^{(j)})$, and $\Omega_{i,j}(\Delta w_m^{(i)}, \Delta w_n^{(j)})$ in a naive manner requires computing the Hessians. However, we point out that the Hessian computation can be avoided. We estimate Ω by directly measuring the change in loss due to network perturbations:

$$\begin{aligned} \Omega_{i,i}(\Delta w_m^{(i)}, \Delta w_m^{(i)}) &\approx 2 \left(\mathcal{L}(w + \Delta w_m^{(i)}) - \mathcal{L}(w) \right) \\ \Omega_{j,j}(\Delta w_n^{(j)}, \Delta w_n^{(j)}) &\approx 2 \left(\mathcal{L}(w + \Delta w_n^{(j)}) - \mathcal{L}(w) \right) \\ \Omega_{i,i}(\Delta w_m^{(i)}, \Delta w_m^{(i)}) + \Omega_{j,j}(\Delta w_n^{(j)}, \Delta w_n^{(j)}) + 2\Omega_{i,j}(\Delta w_m^{(i)}, \Delta w_n^{(j)}) \\ &\approx 2 \left(\mathcal{L}(w + \Delta w_m^{(i)} + \Delta w_n^{(j)}) - \mathcal{L}(w) \right) \end{aligned} \quad (12)$$

Equations in (12) follow from equation (4). The right sides of the three equations in (12) are computed by taking the difference in the loss of the quantized model from that of the full-precision model. Subtracting the first two equations from the third equation:

$$\begin{aligned} \Omega_{i,j}(\Delta w_m^{(i)}, \Delta w_n^{(j)}) &\approx \mathcal{L}(w + \Delta w_m^{(i)} + \Delta w_n^{(j)}) + \mathcal{L}(w) \\ &\quad - \mathcal{L}(w + \Delta w_m^{(i)}) - \mathcal{L}(w + \Delta w_n^{(j)}) \end{aligned} \quad (13)$$

Algorithm 1 CLADO

Input: I -layer network with bit-width options $\mathbb{B} = \{b_1, b_2, \dots, b_{|\mathbb{B}|}\}$ and model size target C_{target} .
Output: The bit-width decision for each layer.
 Evaluate the loss of full-precision model: $\mathcal{L}(w)$
 Initialize $\hat{G} \in \mathbb{R}^{|\mathbb{B}|I \times |\mathbb{B}|I}$
for i in $\{0, \dots, I-1\}$ **do**
 for m in $\{1, \dots, |\mathbb{B}|\}$ **do**
 $\Delta w^{(i)} \leftarrow Q(w^{(i)}, b_m) - w^{(i)}$ // Quantize layer i to b_m bits
 Evaluate loss of quantized network $\mathcal{L}(w + \Delta w^{(i)})$
 $S \leftarrow 2(\mathcal{L}(w + \Delta w^{(i)}) - \mathcal{L}(w))$ // layer-specific sensitivity
 $\hat{G}_{|\mathbb{B}|i+m, |\mathbb{B}|i+m} \leftarrow S$ // Store layer-specific sensitivity
 end for
end for
for i in $\{0, \dots, I-2\}$ **do**
 for j in $\{i+1, \dots, I-1\}$ **do**
 for (m, n) in $\{1, \dots, |\mathbb{B}|\} \times \{1, \dots, |\mathbb{B}|\}$ **do**
 // Quantize layer i to b_m bits and layer j to b_n bits
 $\Delta w^{(i)} \leftarrow Q(w^{(i)}, b_m) - w^{(i)}$
 $\Delta w^{(j)} \leftarrow Q(w^{(j)}, b_n) - w^{(j)}$
 Measure loss of quantized model $\mathcal{L}(w + \Delta w^{(i)} + \Delta w^{(j)})$
 // Compute cross-layer sensitivities using equation (13)
 $S \leftarrow \mathcal{L}(w + \Delta w^{(i)} + \Delta w^{(j)}) - \mathcal{L}(w)$
 $S \leftarrow S - 0.5\hat{G}_{|\mathbb{B}|i+m, |\mathbb{B}|i+m}$
 $S \leftarrow S - 0.5\hat{G}_{|\mathbb{B}|j+n, |\mathbb{B}|j+n}$
 $\hat{G}_{|\mathbb{B}|i+m, |\mathbb{B}|j+n} \leftarrow S$
 end for
 end for
end for
 Compute the eigen-decomposition: $\hat{G} = \sum_{i=1}^{|\mathbb{B}|I} u_i e_i u_i^T$ and set PSD approximated $\hat{G} \leftarrow \sum_{i=1, e_i > 0}^{|\mathbb{B}|I} u_i e_i u_i^T$
 Solve the IQP problem in (11) by plugging in $\hat{G}, \mathbb{B}$, and C_{target} to get the optimal solution α^* .
Return: The bit-width decisions indicated by α^* .

Importantly, (12) and (13) compute all sensitivities by *only forward evaluations* of DNNs on the small sensitivity set. In total, CLADO sensitivity computation requires $\frac{1}{2}|\mathbb{B}|I(|\mathbb{B}|I + 1)$ measurements of the network on a small set of samples. After sensitivity computation, we apply a positive semi-definite (PSD) approximation to the matrix $\hat{G}$. Ideally, G is PSD if computed on the entire training set after full convergence, but using a small sensitivity set can introduce measurement errors, making it indefinite. In practice, PSD approximation is crucial for meaningful MPQ solutions (see Section VII for details).

V. EXPERIMENTAL RESULTS: CNNs

A. Experimental Settings

Dataset and models: We conduct experiments on the ImageNet dataset [26]. To compare CLADO to existing work, we test two state-of-the-art algorithms: MPQCO [4] and HAWQ-V3 [32]. We evaluate with four convolutional models: ResNet-34/50 [14], MobileNetV3 [15], and RegNet-3.2GF [25]. For all models excluding MobileNetV3, we quantize activations to 8-bit and consider three candidate bit-widths for weights: $\mathbb{B} = \{2, 4, 8\}$. (Note that, as shown in Algorithm 1, CLADO can work with any number ($|\mathbb{B}|$) of bit-width candidates, for experiments, we use $|\mathbb{B}| = 3$.) We consider more conservative quantization bit-width candidates $\mathbb{B} = \{4, 6, 8\}$ for MobileNetV3 because of its high parameter efficiency.

Implementation details: All algorithms are implemented in PyTorch [23]. The publicly available package CVXPY [7] is used to solve the IQP problem in CLADO with GUROBI [13] as its backend. For fair comparisons, all algorithms are run using the same procedures and settings. We first download pretrained full-precision models from TorchVision. Then, model quantization is performed

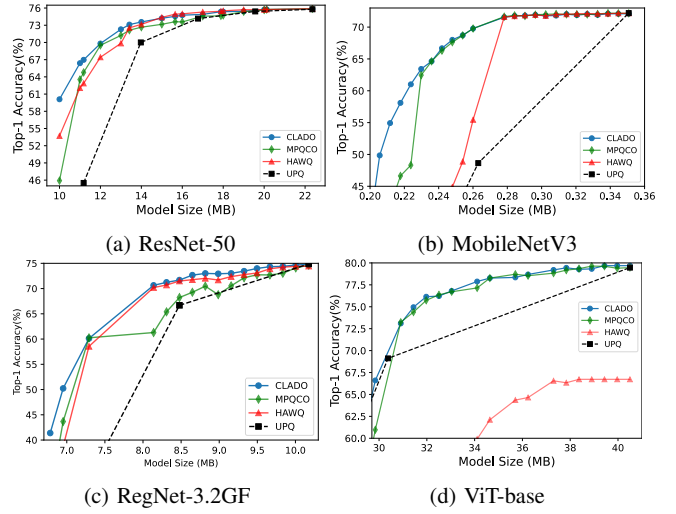


Fig. 2: CNNs and ViT on the ImageNet dataset

using MQBench [19]. Following MPQCO, quantization scale factors (and zero points in the affine case) are determined by minimization of the MSE between the `float32` values and their quantized values. After MQBench quantization, different MPQ algorithms are tested for mixed-precision quantization.

Use of multiple sensitivity sets: Results of sensitivity-based algorithms depend on the samples in the sensitivity set and show some variation. To study the dependence of algorithms on the sensitivity set and to reduce the variation in results, we use multiple sensitivity sets: for each sample size ranging from 256 to 4096, we randomly construct 24 sets with the given size. Then, we test the performance of algorithms on each of the 24 sets.

B. Comparison of Different MPQ Algorithms

Runtime: CLADO requires $\frac{1}{2}|\mathbb{B}|I(|\mathbb{B}|I + 1)$ measurements of the network output on the small sensitivity set. On a single Nvidia RTX2080 GPU, our implementation takes 1 hour to generate the sensitivities for the ResNet-34 and 2.5 hours for the ResNet-50 model. HAWQ takes roughly the same number of GPU-hours as CLADO to compute sensitivities. MPQCO is the fastest one, taking 5-10 minutes for its sensitivity estimation.

Average performance: Table I presents the averaged post-training quantization performance of algorithms under different model sizes (see Figure 2 for tradeoff curves with more data points.) CLADO delivers the best PTQ models under most constraints. Its advantage is especially prominent when models are aggressively compressed to low precisions. For example, under the smallest model size constraints in the table, CLADO demonstrates 5.73% (ResNet-34) to 32% (MobileNetV3) accuracy improvement over HAWQ and MPQCO. However, we note that the PTQ degradation in these low precision regions is too high and renders the results less meaningful. To address this, we apply additional quantization-aware fine-tuning. The QAT results are in Figure 3.

We found that, though CLADO is derived with (13) under the PTQ settings, its bit-width assignments remain superior to others after fine-tuning. Since QAT significantly reduces degradation, the difference between algorithms' after-QAT performance is much smaller. E.g., for 8.87MB ResNet-34, CLADO, HAWQ, MPQCO achieve pre-QAT accuracies of 44.40%, 9.96%, and 0.51%, respectively, and post-QAT accuracies of 72.34%, 72.00%, and 72.08%. *Although the after-QAT difference is small, CLADO maintains meaningful advantage: e.g., for ResNet-34 with 8.87MB size constraint and ResNet-50 with 8.94MB,*

TABLE I: MPQ results (PTQ): CNNs and ViT on ImageNet. Numbers denote the averaged Top-1 accuracy (percentage) for different algorithms. CLADO delivers the best MPQ results under most model sizes. (*: for ablation study, CLADO with cross-layer terms removed. ⁺: experiment using per-channel affine instead of per-tensor symmetric quantization.)

Model	ResNet-34 INT8 size: 20.27MB; Acc: 73.10			ResNet-50 INT8 size: 22.36MB; Acc: 75.87			MobileNetV3-Large ⁺ INT8 size: 0.35MB; Acc: 72.20			RegNet-3.2GF INT8 size: 13.57MB; Acc: 77.88			ViT-base ⁺ INT4 size: 40.5MB; Acc: 79.47		
Size (MB)	10.13	12.16	14.19	10.00	13.42	17.89	0.21	0.25	0.29	6.78	9.50	12.89	29.84	31.97	34.11
HAWQ	56.86	68.13	72.00	53.70	72.55	75.36	15.15	44.88	71.82	33.84	73.08	77.24	10.57	52.44	59.91
MPQCO	53.39	67.51	71.53	45.90	72.13	74.64	17.64	67.62	71.75	29.34	72.73	77.51	60.96	75.73	77.15
CLADO*	54.47	70.37	71.67	53.22	72.51	75.35	43.80	67.52	71.75	32.84	73.48	77.51	54.42	70.75	74.05
CLADO	62.59	70.38	72.20	60.13	73.10	75.42	49.85	68.00	71.81	41.40	73.98	77.53	66.60	76.15	77.88

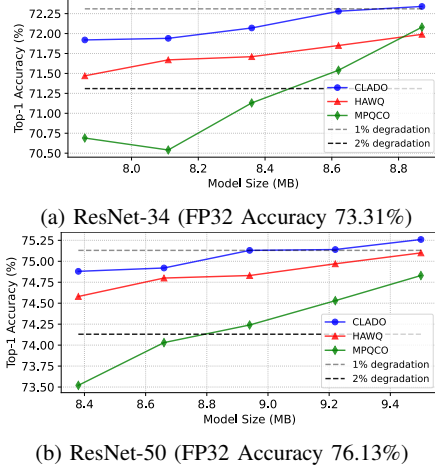


Fig. 3: MPQ results (QAT): QAT fine-tuning based on CLADO outperforms fine-tuning based on other MPQ methods. (Range of model sizes is chosen to be close to 3-bit UPQ. With higher model size, all MPQ algorithms tend to recover FP32 performance.)

9.22MB, and 9.50MB constraints, CLADO maintains $\leq 1\%$ accuracy degradation while others exhibit a higher degradation.

Dependence on the sensitivity set: Figure 4 shows algorithms’ variation due to different samples of the sensitivity set. For each sample size, it presents, the median, the upper quartile (75%), and the lower quartile (25%) of algorithms’ performance on the 24 randomly sampled sensitivity sets. CLADO produces reasonably stable results. Though it may have slightly higher variation sometimes, its lower quartile performance is almost always better than alternatives’ upper quartile performance, which is especially true when the sample size is ≥ 1024 on ImageNet.

Analysis of bit-width assignments: Figure 5 visualizes the MPQ decisions using different algorithms on ResNet-50. Different MPQ algorithms share some commonalities, *e.g.*, generally they assign more bits to shallow layers and less bits to deep layers. This aligns with our observation that shallow layers have higher sensitivities and fewer parameters. However, algorithms decide quite differently for certain layers: compared to HAWQ and MPQCO, CLADO chooses to quantize more aggressively on the first convolutional layer of `layer1.1` (index 4), and `layer4.1` (index 46) and more conservatively on the downsampling layer of `layer3.0` (index 26).

VI. EXPERIMENTAL RESULTS: VISION TRANSFORMER

We also tested our MPQ flow on a transformer-based vision model. The Vision Transformer (ViT) has significantly advanced computer vision by applying transformer architectures, originally designed for natural language processing, to visual data [11]. Its ability to capture long-range dependencies and global context has shown superior performance compared to traditional convolutional neural networks

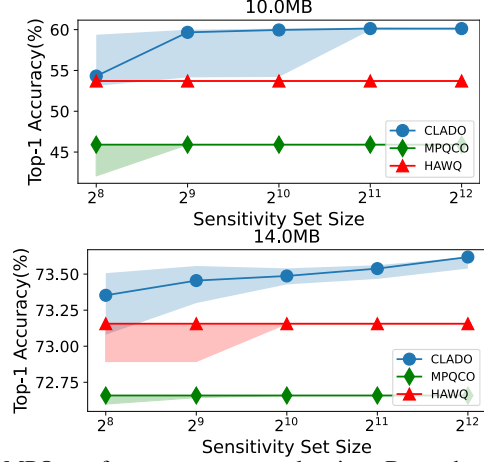


Fig. 4: MPQ performance vs. sample size. Data shows median performance across 24 random sensitivity sets. Colored regions cover the upper and lower performance quartiles.

(CNNs) in tasks like image classification and object detection. ViT is also commonly used in CLIP [24] as an image encoder to extract visual features, enabling zero-shot image classification and connecting vision to text. We use a pretrained ViT-based model from the HuggingFace library [30] and evaluate the quantized model on the ImageNet dataset using various PTQ algorithms [26].

Table I shows the size/performance tradeoffs (see Figure 2d for full Pareto fronts). We found that *CLADO almost always produces the best MPQ solutions*. The advantage of CLADO over the next best alternative is small under large size constraints. Under more restrictive size constraints, CLADO becomes more advantageous.

VII. ABLATION STUDIES

Cross-Layer dependencies: To demonstrate the importance of cross-layer dependencies, we test the performance of CLADO assuming zero cross-layer sensitivities for all layers. We refer to this set of experiments as “CLADO*”. Compared to full CLADO, *ignoring cross-layer dependencies consistently and significantly degrades MPQ performance* (Table I). Cross-layer dependencies seem to be rather prominent for ViT and for CNN compression under small size constraints.

BRECQ [18] claimed that block-level optimization performs best for quantization by adaptive rounding. Interestingly, however, we show that this observation does not extend to MPQ. Following BRECQ, we conducted ablation experiments in which cross-layer dependencies are only considered within blocks of layers. In this set of experiments, we intentionally leave out the inter-block dependencies. We refer to this group of experiments as “block interactions” (the black curves in Figure 6). *We found that ignoring inter-block de-*

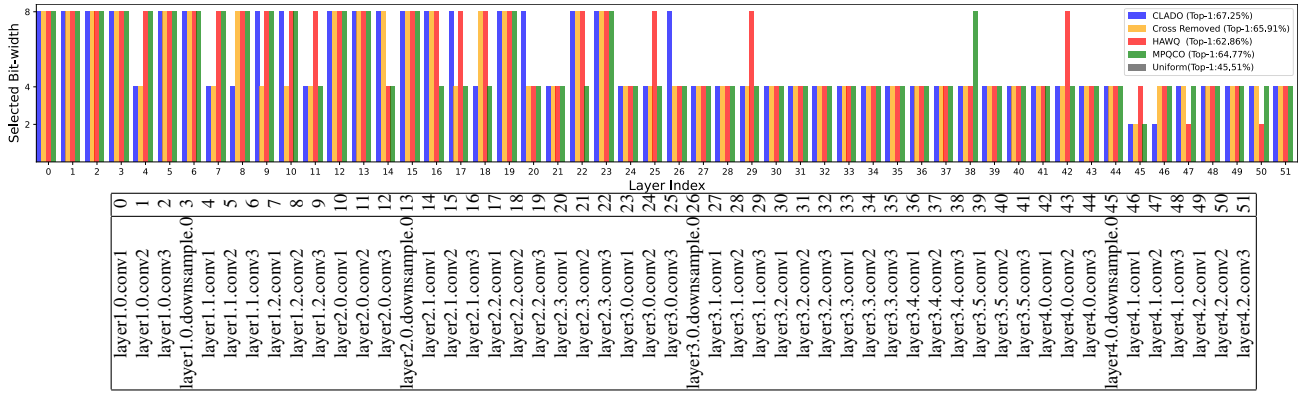
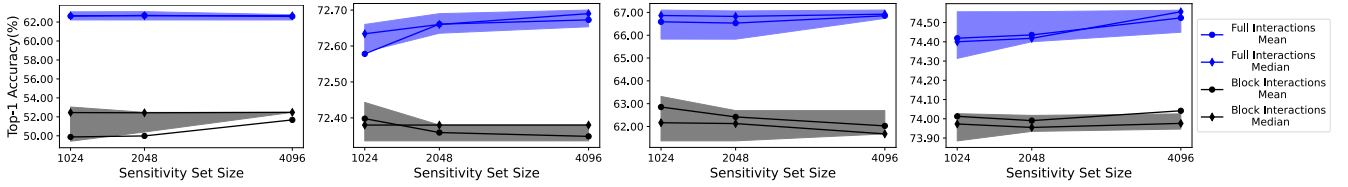


Fig. 5: Bit-width assignments to ResNet-50 along with the layer index-layer name mappings. Model size constraint is set to be 11.18MB, corresponding to 4-bit UPQ.



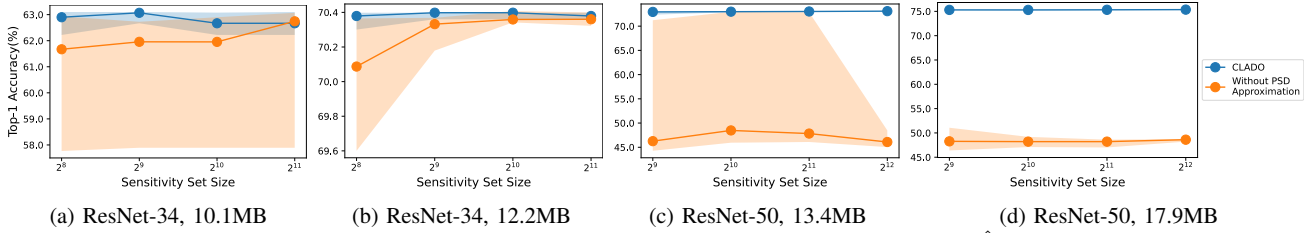
(a) ResNet-34: 10.14MB

(b) ResNet-34: 16.22MB

(c) ResNet-50: 11.18MB

(d) ResNet-50: 15.65MB

Fig. 6: Leaving out inter-block dependencies worsens MPQ. Blue: CLADO, captures all-layer dependencies. Black: inter-block dependencies ignored, following BRECQ [18] Data represents median performance on 24 randomly sampled sensitivity sets. Colored regions cover the upper and lower quartiles of performance.



(a) ResNet-34, 10.1MB

(b) ResNet-34, 12.2MB

(c) ResNet-50, 13.4MB

(d) ResNet-50, 17.9MB

Fig. 7: Ablation on PSD Approximation of the Sensitivity Matrix $\hat{G}$.

TABLE II: Approx. accuracy of the second-order quantization error (vHv): our fast method vs. the exact Hessian evaluation.

ResNet20 Layers	module.layer1 .0.conv1 (2-bit)	module.layer2 .1.conv2 (6-bit)	module.layer3 .2.conv1 (2-bit)	module.layer3 .2.conv1 (4-bit)	module.layer3 .2.conv1 (6-bit)	module.fc (2-bit)	module.fc (4-bit)
vHv (exact)	0.17105	0.00005	0.09384	0.00374	0.00037	0.03154	0.00057
vHv (ours)	0.14670	0.00006	0.09912	0.00362	0.00038	0.03190	0.00057

dependencies significantly worsens the MPQ solution. We hypothesize that MPQ, with far fewer parameters ($|\mathbb{B}|I$) than adaptive rounding, is less prone to the overfitting issue noted by BRECQ. Thus, using full pairwise interactions yields more robust results, while limiting to intra-block interactions likely underfits MPQ solutions.

PSD sensitivity matrix approximation: PSD approximation is critical to CLADO’s superior performance over other studied algorithms. To explore this, we disable the use of PSD approximation in CLADO, keeping the rest of CLADO unchanged. We found that it makes CVXPY unable to converge on a solution in more than 3 hours (with the PSD approximation, the solver computes the optimal solutions within seconds). This observation aligns with our expectation that the convexity of the objective Ω makes the IQP more well-behaved and easier to solve. The results are captured by the orange curves in Figures 7: though PSD does not always produce best results, it improves their consistency, and, in some cases, prevents a severe degradation in the quality of solutions.

Approximation accuracy of our fast Hessian-free method: We evaluate the accuracy of our fast method that avoids the Hessian computation by comparing the quantization-induced loss changes with those from the exact Hessian method, which is $7\times$ slower and requires more CUDA memory. Results for randomly selected shallow and deep layers in ResNet20 are shown in Table II, where v represents the quantization error at each bit-width. The approximation error is generally small.

VIII. CONCLUSIONS

We present CLADO, a sensitivity-based mixed-precision quantization algorithm that exploits cross-layer quantization error dependencies. We derive a sensitivity objective from 2^{nd} -order Taylor expansion of loss, and reformulate the bit-width assignment problem as an integer quadratic program that can be solved efficiently. We devise an efficient algorithm to compute the cross-layer dependencies that requires only forward evaluations of networks. Extensive experiments are conducted to demonstrate the effectiveness of the proposed method.

REFERENCES

- [1] S. Azizi, M. Nazemi, A. Fayyazi, and M. Pedram. Automated optimization of deep neural networks: Dynamic bit-width and layer-width selection via cluster-based parzen estimation. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024.
- [2] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer. ZeroQ: A novel zero shot quantization framework. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13166–13175, 2020.
- [3] A. Chauhan, U. Tiwari, and V. N. R. Post training mixed precision quantization of neural networks using first-order information. *IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 1335–1344, 2023.
- [4] W. Chen, P. Wang, and J. Cheng. Towards mixed-precision quantization of neural networks via constrained optimization. *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5330–5339, 2021.
- [5] X. Chu, L. Li, and B. Zhang. Make repvgg greater again: A quantization-aware approach. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38:11624–11632, 2024.
- [6] M. Courbariaux, Y. Bengio, and J.-P. David. BinaryConnect: Training deep neural networks with binary weights during propagations. *Advances in neural information processing systems*, 28, 2015.
- [7] S. Diamond and S. P. Boyd. Cvxpy: A python-embedded modeling language for convex optimization. *Journal of machine learning research : JMLR*, 17, 2016.
- [8] P. Dong, L. Li, Z. Wei, X.-Y. Niu, Z. Tian, and H. Pan. EMQ: Evolving training-free proxies for automated mixed precision quantization. *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17030–17040, 2023.
- [9] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ-V2: Hessian aware trace-weighted quantization of neural networks. *ArXiv*, abs/1911.03852, 2020.
- [10] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 293–302, 2019.
- [11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ArXiv*, abs/2010.11929, 2020.
- [12] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun. Single path one-shot neural architecture search with uniform sampling. In *ECCV*, 2020.
- [13] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023.
- [14] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2015.
- [15] A. G. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam. Searching for mobilenetv3. *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324, 2019.
- [16] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer. Integer-only bert quantization. *ArXiv*, abs/2101.01321, 2021.
- [17] S. Kundu, S. Wang, Q. Sun, P. A. Bearel, and M. Pedram. BMPQ: bit-gradient sensitivity-driven mixed-precision quantization of dnns from scratch. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 588–591, 2022.
- [18] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu. BRECQ: Pushing the limit of post-training quantization by block reconstruction. *ArXiv*, abs/2102.05426, 2021.
- [19] Y. Li*, M. Shen*, J. Ma*, Y. Ren*, M. Zhao*, Q. Zhang*, R. Gong*, F. Yu, and J. Yan. Mqbench: Towards reproducible and deployable model quantization benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [20] Q. Lou, F. Guo, L. Liu, M. Kim, and L. Jiang. AutoQ: Automated kernel-wise neural network quantization. *ArXiv*, abs/1902.05690, 2020.
- [21] Y. Ma, T. Jin, X. Zheng, Y. Wang, H. Li, G. Jiang, W. Zhang, and R. Ji. OMPQ: Orthogonal mixed precision quantization. *ArXiv*, abs/2109.07865, 2021.
- [22] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort. Up or down? adaptive rounding for post-training quantization. *ArXiv*, abs/2004.10568, 2020.
- [23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [24] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [25] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár. Designing network design spaces. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10425–10433, 2020.
- [26] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115:211–252, 2014.
- [27] C. Tang, K. Ouyang, Z. Wang, Y. Zhu, Y. Wang, W. Ji, and W. Zhu. Mixed-precision neural network quantization via learned layer-wise importance. *ArXiv*, abs/2203.08368, 2022.
- [28] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han. HAQ: Hardware-aware automated quantization with mixed precision. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8604–8612, 2019.
- [29] X. Wei, R. Gong, Y. Li, X. Liu, and F. Yu. QDrop: Randomly dropping quantization for extremely low-bit post-training quantization. *ArXiv*, abs/2203.05740, 2022.
- [30] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the conference on empirical methods in natural language processing: system demonstrations*, pages 38–45, 2020.
- [31] B. Wu, Y. Wang, P. Zhang, Y. Tian, P. Vajda, and K. Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search. *ArXiv*, abs/1812.00090, 2018.
- [32] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. Mahoney, et al. HAWQ-V3: Dyadic neural network quantization. In *International Conference on Machine Learning*, pages 11875–11886. PMLR, 2021.